

Documentación del Proyecto

Autor: César Campos - Diego Guillen

Fecha: 08/10/2025

Species Whisper

Species Whisper es una aplicación de inteligencia artificial para el **reconocimiento de cantos de aves** mediante grabaciones de audio.

Su principal objetivo es **funcionar de manera local**, permitiendo identificar especies incluso en **lugares sin acceso a internet**.

Objetivo del Proyecto

Desarrollar un sistema que identifique especies de aves a partir de sonidos grabados por el usuario, utilizando técnicas de **procesamiento de audio** y **modelos de aprendizaje profundo**.

La aplicación busca apoyar iniciativas de conservación, educación ambiental y monitoreo de biodiversidad en regiones rurales.

Tecnologías Principales

| Tipo | Tecnologías |

|-----|-----|

| **Framework Web** | Flask |

| **IA y Deep Learning** | TensorFlow, Keras |

| **Procesamiento de Audio** | Librosa, SoundFile, NumPy, SciPy |

| **Aprendizaje Automático** | Scikit-learn |

| **Servicios y API** | FastAPI, Uvicorn |

| **Audio y Multimedia** | FFmpeg, Pygame |

| **Documentación** | FPDF |

| **Gestión de dependencias** | requirements.in → requirements.txt |

🌀 Instalación y Ejecución

Requisitos previos

- Python 3.10 o superior
- pip y virtualenv
- FFmpeg instalado en el sistema

Configuración local

```
```bash
```

```
Clonar el repositorio
```

```
git clone https://github.com/usuario/species_whisper.git
```

```
cd species_whisper
```

```
Crear y activar entorno virtual
```

```
python -m venv venv
```

```
source venv/bin/activate # En Windows: venv\Scripts\activate
```

```
Instalar dependencias
```

```
pip install -r requirements.txt
```

```
Ejecutar la aplicación Flask
```

```
python app.py
```

```
...
```

La aplicación estará disponible en:

**\*\*http://127.0.0.1:5000\*\***

---

## **# # Pruebas Unitarias**

Las pruebas se ejecutan por módulo utilizando **\*\*pytest\*\***.

**```bash**

**# Prueba para la aplicación principal**

**pytest app.py**

**# Prueba para la carga de modelos**

**pytest load\_model.py**

\\\

---

## ## Documentación de la Aplicación Flask

Esta aplicación permite **identificar aves mediante sus cantos** a través de una interfaz web sencilla.

### ### 1 Descripción general

- Al ejecutar `app.py`, se despliega un servidor local en `http://127.0.0.1:5000`.
- La página inicial muestra una interfaz para **cargar archivos de audio** y realizar la identificación.

### ### 2 Uso de la interfaz

## 1. **Cargar audio**

- Hacer clic en **"Elegir archivo"** y seleccionar el canto a identificar.

## 2. **Opciones después de cargar el audio**

- **Escuchar el canto** usando el reproductor de la interfaz.
- **Identificar ave**: al hacer clic, Flask llama al endpoint:

...

`http://127.0.0.1:5000/identify`

...

## 3. **Proceso interno**

- El audio se guarda en `app/statics/media/audio``.
- `integrator.py`` se ejecuta secuencialmente:

### 1. Procesar el audio



2. Cargar el modelo entrenado

3. Identificar la especie de ave

- El nombre en inglés de la especie identificada se devuelve a `app.py`, que busca los **datos correspondientes** (imagen, mapa y descripción) para mostrarlos en pantalla.

4. **Repetir proceso**

- Para identificar otro canto, basta con hacer clic nuevamente en **"Elegir archivo"** y repetir el flujo.

### 3 Ejemplos visuales

- Interfaz inicial (pantalla para cargar audio):

![Interfaz inicial](reports\flask\inicio.png)

- Resultado de identificación (con especie, imagen y mapa):

![Resultado de identificación](reports\flask\prueba.png)

---

## Docker (implementación sugerida)

```bash

Construir imagen

docker build -t species_whisper .

Ejecutar contenedor

```
docker run -p 5000:5000 species_whisper
```

...

Makefile

El proyecto incluye un **Makefile** para automatizar las tareas más comunes del entorno de desarrollo.

Comandos disponibles

| Comando | Descripción |
|---------|-------------|
|---------|-------------|

| | |
|-------|-------|
| ----- | ----- |
|-------|-------|

| | |
|----------------|---|
| `make install` | Crea un entorno virtual e instala las dependencias. |
|----------------|---|

| ``make run`` | Ejecuta la aplicación Flask localmente. |

| ``make test`` | Ejecuta las pruebas unitarias (``pytest app.py`` y ``pytest load_model.py``). |

| ``make docker-build`` | Construye la imagen Docker ``species_whisper``. |

| ``make docker-run`` | Ejecuta el contenedor Docker en el puerto 5000. |

| ``make clean`` | Elimina archivos temporales y entornos virtuales. |

| ``make help`` | Muestra el resumen de comandos disponibles. |

Esto permite gestionar el ciclo completo del proyecto sin recordar los comandos largos.

Ejemplo:

```
```bash
```

```
make docker-build
```

```
make docker-run
```

---

## ## Estructura del Proyecto (sugerida)

...

species\_whisper/

├─ app.py           # Aplicación Flask

├─ load\_model.py     # Carga y predicción del modelo de IA

├─ static/           # Recursos estáticos (iconos, CSS, JS)

├─ templates/        # Vistas HTML

├─ models/           # Modelos entrenados

├─ tests/            # Pruebas unitarias

├─ requirements.in

└─ requirements.txt

└─ README.md

...

---

## ## Autores

- **Diego Guillen**

- **César Campos**

---

## ## Licencia

Este proyecto está licenciado bajo la **\*\*GNU General Public License v3.0 (GPLv3)\*\***.

Puedes usar, modificar y distribuir este software libremente bajo los términos de esta licencia,

siempre y cuando cualquier distribución o versión modificada también sea de código abierto.

Consulta el archivo [LICENSE](LICENSE) para más información.

---

## **## Futuro del Proyecto**

- Implementar un modelo más ligero para dispositivos de bajo consumo.

- Crear una versión móvil offline.
- Añadir identificación por imágenes y sonidos combinados.



## Resultados de Pruebas Unitarias

=====	test	session	starts
=====			
platform win32 -- Python 3.12.2, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\CESAR CAMPOS\species_whisper\.venv\Scripts\python.exe			
cachedir: .pytest_cache			
rootdir: C:\Users\CESAR CAMPOS\species_whisper			
plugins: anyio-4.11.0			
collecting ... collected 2 items			
tests/test_app.py::test_home_route PASSED		[ 50%]	
tests/test_integrator.py::test_get_species_prediction PASSED		[100%]	
=====	2	passed	in 6.30s
=====			